

Lecture Notes: DeepSeek-R1-Zero and Group Relative Policy Optimization

NLP / LLMs Lecture Handout

Lecture context

Target audience. Senior undergraduate and graduate students who have seen language modeling and basic machine learning optimization. The lecture assumes familiarity with token probabilities, cross-entropy training, and basic gradients, but it reviews the reinforcement-learning notation needed for GRPO.

Time budget. A 90-minute lecture can be organized as follows:

Time	Topic
0–10 min	Motivation: why post-train LLMs for reasoning?
10–20 min	DeepSeek-R1-Zero pipeline overview.
20–35 min	From language-model policy gradients to PPO-style clipped updates.
35–55 min	GRPO data flow and objective, step by step.
55–65 min	Rule-based rewards: accuracy, code tests, and format.
65–75 min	Full symbolic batch walkthrough.
75–83 min	Comparison: PPO vs. GRPO.
83–90 min	Short contrast with the full DeepSeek-R1 pipeline and takeaways.

Learning goals. By the end, students should be able to:

- (1) describe the DeepSeek-R1-Zero post-training pipeline;
- (2) write the GRPO objective at the sequence and token levels;
- (3) explain how a batch of prompts becomes rewards, normalized advantages, and gradient updates;
- (4) explain why GRPO avoids a learned critic/value model;
- (5) contrast DeepSeek-R1-Zero with the fuller DeepSeek-R1 training pipeline.

1 High-level picture

DeepSeek-R1-Zero is a reasoning model obtained by applying large-scale reinforcement learning directly to a base LLM, without first doing supervised fine-tuning on human-written reasoning

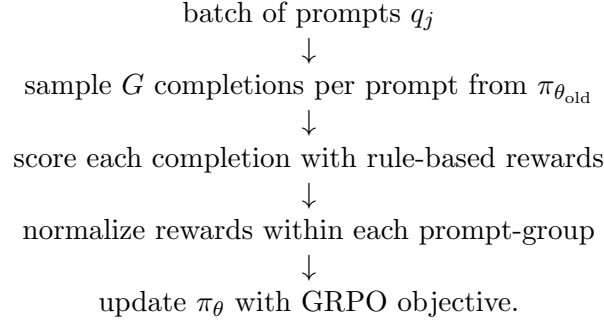
traces. The important conceptual point is that the training signal is not “imitate this reasoning trace.” Instead, it is closer to:

generate several candidate solutions $\longrightarrow$ score them by rules $\longrightarrow$ increase probability of the relatively su

The simplified R1-Zero pipeline is:

$$\boxed{\text{DeepSeek-V3-Base}} \xrightarrow{\text{RL with GRPO}} \boxed{\text{DeepSeek-R1-Zero}}.$$

The central training loop is:



In the reported R1-Zero training details, each question used $G = 16$ sampled outputs. A training step used 32 unique questions, so the corresponding group-expanded batch size was $32 \times 16 = 512$ completions. The reported setup also used learning rate 3×10^{-6} , KL coefficient 0.001, rollout temperature 1, and very long maximum generation lengths. See [2].

2 Notation

Let

- $q \in \mathcal{Q}$ be a reasoning prompt, for example a math, coding, science, or logic problem;
- $x = T(q)$ be the formatted model input after applying the training template;
- $o = (y_1, \dots, y_T)$ be a generated completion;
- $\pi_{\theta}(o \mid x)$ be the LLM policy, i.e. the probability of completion o given input x ;
- θ_{old} be the frozen rollout policy used to sample the current batch;
- π_{ref} be the reference policy used in the KL penalty;
- G be the group size, i.e. the number of completions sampled per prompt. For R1-Zero, use $G = 16$ in lecture unless you are discussing variants.

Because an LLM is autoregressive,

$$\pi_{\theta}(o \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}), \quad \log \pi_{\theta}(o \mid x) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (1)$$

For a mini-batch with B unique prompts, write:

$$\mathcal{B} = \{q_j\}_{j=1}^B, \quad \mathcal{G}_j = \{o_{j,1}, \dots, o_{j,G}\}. \quad (2)$$

For R1-Zero-style lecture notation, set $B = 32$ and $G = 16$ when discussing reported training scale, but keep the derivation general.

3 Training template

The model is prompted to produce a reasoning portion followed by a final answer. For mathematical notation, it is useful to split a completion into

$$o = (z, a),$$

where z denotes reasoning tokens and a denotes the final answer tokens. In class, avoid spending time on long reasoning traces; use z as a symbolic object.

A simplified template is:

$$T(q) = \text{User: } q \text{ Assistant: } \langle \text{think} \rangle z \langle / \text{think} \rangle \langle \text{answer} \rangle a \langle / \text{answer} \rangle.$$

More carefully, $T(q)$ is the input prefix, and the model generates the tokens that should fill the reasoning and answer regions. The format reward checks whether the generated sequence respects this structural convention.

4 Step-by-step data flow

4.1 Step 1: sample a batch of prompts

Sample prompts from the RL prompt distribution:

$$q_1, \dots, q_B \sim \mathcal{D}_{\text{RL}}. \quad (3)$$

The dataset is not a set of target reasoning traces. It is a set of prompts for which we can usually compute a reward. For example:

- math prompts with known final answers;
- coding prompts with unit tests;
- logic/science prompts with verifiable targets.

4.2 Step 2: sample a group of completions for each prompt

For each prompt q_j , construct $x_j = T(q_j)$ and sample G outputs from the old policy:

$$o_{j,i} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x_j), \quad i = 1, \dots, G. \quad (4)$$

Equivalently, at the token level:

$$o_{j,i} = (y_{j,i,1}, \dots, y_{j,i,T_{j,i}}), \quad (5)$$

where

$$y_{j,i,t} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x_j, y_{j,i,<t}). \quad (6)$$

The key GRPO idea is that the G completions for the same prompt form a comparison group. For R1-Zero, use

$$G = 16. \quad (7)$$

So one prompt generates 16 candidate solutions.

4.3 Step 3: compute rule-based rewards

Each completion receives a scalar reward:

$$r_{j,i} = R(q_j, o_{j,i}). \quad (8)$$

For lecture purposes, decompose it as:

$$R(q, o) = w_{\text{acc}} R_{\text{acc}}(q, o) + w_{\text{fmt}} R_{\text{fmt}}(o), \quad (9)$$

where $w_{\text{acc}}, w_{\text{fmt}} \geq 0$ are teaching notation for reward weights. The exact implementation details and weights need not be the same across reproductions.

Accuracy reward for math. Let $a^*(q)$ be the ground-truth answer and $\text{extract}(o)$ be a parser that extracts the final answer from the completion. A simple binary accuracy reward is

$$R_{\text{acc}}^{\text{math}}(q, o) = \mathbf{1}\{\text{extract}(o) = a^*(q)\}. \quad (10)$$

One can also use partial or symbolic equivalence checking:

$$R_{\text{acc}}^{\text{math}}(q, o) = \mathbf{1}\{\text{normalize}(\text{extract}(o)) = \text{normalize}(a^*(q))\}. \quad (11)$$

Accuracy reward for code. For a coding task with tests $\mathcal{T}(q) = \{\tau_1, \dots, \tau_K\}$, define

$$R_{\text{acc}}^{\text{code}}(q, o) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}\{\text{run}(o, \tau_k) = \text{pass}\}. \quad (12)$$

The all-or-nothing version is

$$R_{\text{acc}}^{\text{code}}(q, o) = \prod_{k=1}^K \mathbf{1}\{\text{run}(o, \tau_k) = \text{pass}\}. \quad (13)$$

Format reward. A simple format reward is

$$R_{\text{fmt}}(o) = \mathbf{1}\{o \text{ contains well-formed reasoning and answer tags}\}. \quad (14)$$

For example, one could check whether the output has the schematic form

$$\text{<think> } z \text{ </think> <answer> } a \text{ </answer>}.$$

The R1-Zero paper emphasizes rule-based rewards, mainly accuracy rewards and format rewards, and does not use a neural outcome or process reward model for R1-Zero. The paper states that neural reward models can introduce reward-hacking concerns and extra training complexity; see [1].

4.4 Step 4: compute group-relative advantages

For each prompt q_j , we have G rewards:

$$r_{j,1}, r_{j,2}, \dots, r_{j,G}. \quad (15)$$

Compute the group mean:

$$\mu_j = \frac{1}{G} \sum_{i=1}^G r_{j,i}, \quad (16)$$

and group standard deviation:

$$\sigma_j = \sqrt{\frac{1}{G} \sum_{i=1}^G (r_{j,i} - \mu_j)^2 + \delta}, \quad (17)$$

where $\delta > 0$ is a small numerical constant used in implementations to avoid division by zero.

Then define the group-relative advantage:

$$A_{j,i} = \frac{r_{j,i} - \mu_j}{\sigma_j}. \quad (18)$$

This says:

- completions that are better than the group average have $A_{j,i} > 0$;
- completions that are worse than the group average have $A_{j,i} < 0$;
- completions near the group average have $A_{j,i} \approx 0$.

A useful classroom identity is:

$$\frac{1}{G} \sum_{i=1}^G A_{j,i} = 0. \quad (19)$$

So, for each prompt, GRPO turns absolute reward into within-prompt relative reward.

Why normalize within prompt? Prompt difficulty varies. One prompt may be easy, so many completions are correct; another may be hard, so most are wrong. The group baseline compares candidates for the same prompt, rather than comparing raw scores across unrelated prompts.

Important edge case. If all G completions receive the same reward, then $\sigma_j \approx 0$ and there is essentially no relative learning signal for that prompt. For example, if all 16 completions are wrong and all receive zero reward, then GRPO cannot tell which completion was better.

5 From policy gradient to GRPO

5.1 Plain policy-gradient objective

The ideal RL objective for a language-model policy is

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}_{\text{RL}}, o \sim \pi_\theta(\cdot | T(q))} [R(q, o)]. \quad (20)$$

Using the score-function identity,

$$\nabla_\theta J(\theta) = \mathbb{E}_{q,o} [R(q, o) \nabla_\theta \log \pi_\theta(o | T(q))] \quad (21)$$

$$= \mathbb{E}_{q,o} \left[R(q, o) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t | T(q), y_{<t}) \right]. \quad (22)$$

A baseline $b(q)$ can be subtracted without changing the expected gradient, as long as it does not depend on the sampled action/completion:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{q,o} [(R(q, o) - b(q)) \nabla_{\theta} \log \pi_{\theta}(o \mid T(q))]. \quad (23)$$

Traditional actor-critic methods learn a value function to supply this baseline. GRPO avoids a learned value model and instead uses group rewards from multiple completions for the same prompt.

5.2 Why PPO-style ratios appear

The completions in the batch were sampled from $\pi_{\theta_{\text{old}}}$, but we want to update π_{θ} . Therefore we use an importance ratio. At the token level, define

$$\rho_{j,i,t}(\theta) = \frac{\pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t})}{\pi_{\theta_{\text{old}}}(y_{j,i,t} \mid x_j, y_{j,i,<t})}. \quad (24)$$

A naive surrogate would increase

$$\rho_{j,i,t}(\theta) A_{j,i}. \quad (25)$$

But if ρ becomes too large or too small, the update can become unstable. PPO-style clipping replaces this with

$$\ell_{j,i,t}^{\text{clip}}(\theta) = \min(\rho_{j,i,t}(\theta) A_{j,i}, \text{clip}(\rho_{j,i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) A_{j,i}), \quad (26)$$

where $\varepsilon > 0$ is the clipping hyperparameter.

Interpretation of clipping. If $A_{j,i} > 0$, the model wants to increase the probability of the tokens in completion $o_{j,i}$, but the clipped term limits how much probability can increase in one update. If $A_{j,i} < 0$, the model wants to decrease the probability of those tokens, but the clipped term limits how much probability can decrease.

6 KL regularization

LLM post-training often adds a KL penalty so the policy does not move too far from a reference policy. In GRPO, this penalty is placed directly in the objective rather than being folded into the reward.

For a sampled token $y_{j,i,t}$, define

$$\kappa_{j,i,t}(\theta) = \frac{\pi_{\text{ref}}(y_{j,i,t} \mid x_j, y_{j,i,<t})}{\pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t})}. \quad (27)$$

A common single-sample estimator for the KL contribution is

$$d_{j,i,t}^{\text{KL}}(\theta) = \kappa_{j,i,t}(\theta) - \log \kappa_{j,i,t}(\theta) - 1. \quad (28)$$

Since $u - \log u - 1 \geq 0$ for $u > 0$, this term is nonnegative. Its expectation under $y \sim \pi_{\theta}$ estimates

$$D_{\text{KL}}(\pi_{\theta}(\cdot \mid x_j, y_{<t}) \parallel \pi_{\text{ref}}(\cdot \mid x_j, y_{<t})). \quad (29)$$

The reported R1-Zero training details used KL coefficient $\beta = 0.001$ and replaced the reference model with the latest policy every 400 steps [2]. In lecture, emphasize the distinction:

- $\pi_{\theta_{\text{old}}}$: the policy that sampled the current batch; appears in the PPO-style ratio.
- π_{ref} : the policy used as a KL anchor; appears in the regularizer.

7 GRPO objective

Putting the pieces together, the token-level GRPO maximization objective for one batch is

$$\hat{J}_{\text{GRPO}}(\theta) = \frac{1}{B} \sum_{j=1}^B \frac{1}{G} \sum_{i=1}^G \frac{1}{T_{j,i}} \sum_{t=1}^{T_{j,i}} \left[\ell_{j,i,t}^{\text{clip}}(\theta) - \beta d_{j,i,t}^{\text{KL}}(\theta) \right]. \quad (30)$$

Equivalently, in expectation form:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{q \sim \mathcal{D}_{\text{RL}} \\ \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | T(q))}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\ell_{i,t}^{\text{clip}}(\theta) - \beta d_{i,t}^{\text{KL}}(\theta) \right) \right]. \quad (31)$$

In code, one often minimizes the loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\hat{J}_{\text{GRPO}}(\theta). \quad (32)$$

7.1 Gradient intuition

Ignore clipping and KL for a moment, and suppose $\theta = \theta_{\text{old}}$ at the start of the update. Then $\rho_{j,i,t}(\theta) = 1$, and

$$\nabla_{\theta} [\rho_{j,i,t}(\theta) A_{j,i}] = A_{j,i} \nabla_{\theta} \rho_{j,i,t}(\theta) \quad (33)$$

$$= A_{j,i} \rho_{j,i,t}(\theta) \nabla_{\theta} \log \pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t}) \quad (34)$$

$$= A_{j,i} \nabla_{\theta} \log \pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t}). \quad (35)$$

Thus:

- if $A_{j,i} > 0$, gradient ascent increases the probability of all tokens in completion $o_{j,i}$;
- if $A_{j,i} < 0$, gradient ascent decreases the probability of all tokens in completion $o_{j,i}$;
- if $A_{j,i} \approx 0$, the completion contributes little policy-gradient signal.

This is why the method is easy to explain: GRPO rewards outputs that are better than sibling outputs for the same prompt.

8 Symbolic batch walkthrough

Let the batch contain B prompts:

$$\mathcal{B} = \{q_1, \dots, q_B\}. \quad (36)$$

For each prompt q_j , sample $G = 16$ completions:

$$\mathcal{G}_j = \{o_{j,1}, \dots, o_{j,16}\}. \quad (37)$$

The complete rollout table is:

Prompt	Completion	Reward	Advantage
q_1	$o_{1,1}$	$r_{1,1}$	$A_{1,1}$
q_1	$o_{1,2}$	$r_{1,2}$	$A_{1,2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
q_1	$o_{1,16}$	$r_{1,16}$	$A_{1,16}$
q_2	$o_{2,1}$	$r_{2,1}$	$A_{2,1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
q_B	$o_{B,16}$	$r_{B,16}$	$A_{B,16}$

For each row, compute token log-probabilities under three policies:

$$\log \pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t}) \quad \text{current trainable policy,} \quad (38)$$

$$\log \pi_{\theta_{\text{old}}}(y_{j,i,t} \mid x_j, y_{j,i,<t}) \quad \text{rollout policy,} \quad (39)$$

$$\log \pi_{\text{ref}}(y_{j,i,t} \mid x_j, y_{j,i,<t}) \quad \text{reference policy.} \quad (40)$$

Then compute:

$$\rho_{j,i,t}(\theta) = \exp(\log \pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t}) - \log \pi_{\theta_{\text{old}}}(y_{j,i,t} \mid x_j, y_{j,i,<t})), \quad (41)$$

$$\kappa_{j,i,t}(\theta) = \exp(\log \pi_{\text{ref}}(y_{j,i,t} \mid x_j, y_{j,i,<t}) - \log \pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t})), \quad (42)$$

$$d_{j,i,t}^{\text{KL}}(\theta) = \kappa_{j,i,t}(\theta) - \log \kappa_{j,i,t}(\theta) - 1. \quad (43)$$

Finally plug these into Eq. (30) and update:

$$\theta \leftarrow \theta + \eta \nabla_{\theta} \hat{J}_{\text{GRPO}}(\theta) \quad (44)$$

for learning rate η .

9 Pseudocode

1. Initialize policy π_{θ} from the base model.
2. Set $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ for rollout generation.
3. Set or periodically refresh the reference policy π_{ref} .
4. Repeat:
 - (a) Sample prompts $\{q_j\}_{j=1}^B$.
 - (b) For each q_j , sample $G = 16$ completions $\{o_{j,i}\}_{i=1}^G$ from $\pi_{\theta_{\text{old}}}$.
 - (c) Compute rule-based rewards $r_{j,i} = R(q_j, o_{j,i})$.
 - (d) For each prompt j , compute μ_j , σ_j , and $A_{j,i}$.
 - (e) For each token, compute the policy ratio $\rho_{j,i,t}$ and KL estimate $d_{j,i,t}^{\text{KL}}$.
 - (f) Maximize $\hat{J}_{\text{GRPO}}$, or equivalently minimize $-\hat{J}_{\text{GRPO}}$.
 - (g) Refresh $\pi_{\theta_{\text{old}}}$ for new rollouts.

10 Why GRPO is a simpler post-training pipeline

10.1 PPO-style RLHF

A simplified PPO-style LLM post-training system often has:

- a policy model π_θ ;
- a reference model π_{ref} ;
- a reward model R_ψ ;
- a value model or critic V_ϕ ;
- advantage estimation, often with GAE;
- PPO clipping and KL regularization.

The critic is expensive because it is usually another neural model operating over long token sequences.

10.2 GRPO

GRPO keeps the policy, old policy, and reference policy, but removes the learned critic:

No learned V_ϕ .

Instead, it estimates a prompt-specific baseline from sibling completions:

$$\text{critic baseline } V_\phi(q, o_{<t}) \rightsquigarrow \text{group baseline } \mu_j = \frac{1}{G} \sum_{i=1}^G r_{j,i}. \quad (45)$$

This is the main computational simplification. DeepSeekMath introduced GRPO as a PPO variant that foregoes the critic model and estimates the baseline from group scores, reducing training resources [3]. DeepSeek-R1-Zero then uses GRPO for large-scale reasoning RL directly from the base model [1].

11 PPO versus GRPO

Aspect	PPO-style RLHF	GRPO
Baseline / advantage	Learned value model V_ϕ and GAE-style advantage	Group-relative reward normalization
Extra critic model?	Yes	No
Prompt handling	Advantage may depend on token state through critic	G sibling completions for same prompt define relative score
Reward type	Often neural reward model plus KL penalty	R1-Zero mainly rule-based accuracy and format rewards
Credit assignment	Potentially token-level via critic	Same sequence-level advantage assigned to all completion tokens
Main cost	Policy + reward model + critic training/inference	Many rollouts per prompt; long generations
Main simplification	—	Removes critic and avoids neural reward model in R1-Zero

PPO itself is a policy-gradient method that alternates between sampling data and optimizing a surrogate objective, with clipping used to keep updates controlled [4]. GRPO inherits this PPO-style clipping idea but changes how the advantage is estimated.

12 What the model learns

The training objective does not explicitly say:

“reflect,” “verify,” or “think longer.”

Rather, it rewards completions that solve verifiable problems and satisfy the required format. If longer reasoning, self-checking, or re-planning increases correctness, then those behaviors can become more probable under the policy.

This gives a useful teaching slogan:

R1-Zero does not imitate reasoning traces; it is incentivized to discover useful reasoning behavior.

The R1-Zero paper reports emergent behaviors such as self-verification, reflection, and longer chain-of-thought-like computation during RL, but also reports drawbacks such as poor readability and language mixing [1].

13 Limitations and failure modes

- 1. Verifiable rewards are required.** R1-Zero-style RL is most natural when correctness can be checked. Math and code fit this setting better than open-ended writing.
- 2. Sparse rewards can give no signal.** If all 16 completions are wrong, then all rewards may be equal. The group-relative advantage is then zero or numerically unstable. This means sampling diversity and problem difficulty matter.
- 3. Coarse credit assignment.** A single scalar reward gives the same advantage to every token in a completion. Good and bad intermediate steps are not separately labeled.
- 4. Reward hacking remains possible.** Even rule-based rewards can be gamed if the parser, unit tests, or format checker are weak.
- 5. Rollout cost is high.** Sampling $G = 16$ long completions per prompt is expensive, especially when maximum generation length is tens of thousands of tokens.
- 6. Readability is not guaranteed.** R1-Zero can learn to solve problems while producing outputs that are difficult for humans to read. This motivates the full DeepSeek-R1 pipeline.

14 Short contrast: DeepSeek-R1-Zero versus DeepSeek-R1

14.1 DeepSeek-R1-Zero

$$\boxed{\text{Base model}} \xrightarrow{\text{GRPO RL, rule-based rewards}} \boxed{\text{R1-Zero}}. \quad (46)$$

Properties:

- no preliminary supervised fine-tuning stage;
- emphasizes pure RL self-evolution;
- uses rule-based rewards for verifiable reasoning;
- shows strong reasoning but has readability and language-mixing issues.

14.2 DeepSeek-R1

DeepSeek-R1 adds a more elaborate multi-stage pipeline:

$$\begin{array}{c}
\text{Base model} \\
\downarrow \text{cold-start SFT on thousands of long-CoT examples} \\
\text{initial reasoning actor} \\
\downarrow \text{reasoning-oriented RL} \\
\text{RL checkpoint} \\
\downarrow \text{rejection sampling + SFT data construction} \\
\text{SFT model on reasoning and non-reasoning data} \\
\downarrow \text{final RL for reasoning, helpfulness, and harmlessness} \\
\text{DeepSeek-R1.}
\end{array} \tag{47}$$

According to the R1 paper, the full R1 pipeline uses thousands of cold-start examples, then reasoning-oriented RL, then rejection sampling to collect about 600k reasoning samples plus about 200k non-reasoning samples, then another SFT stage and a final RL stage across broader scenarios [1]. The key difference is that R1-Zero is the clean experiment showing that reasoning can be incentivized directly through RL, while R1 is the more practical pipeline for a readable and generally useful assistant.

15 Board-work derivation

A compact derivation suitable for the board:

Step A: Start from expected reward.

$$J(\theta) = \mathbb{E}_{q,o \sim \pi_\theta} [R(q, o)]. \tag{48}$$

Step B: Policy gradient.

$$\nabla J(\theta) = \mathbb{E}[R(q, o) \nabla \log \pi_\theta(o \mid q)]. \tag{49}$$

Step C: Add a baseline.

$$\nabla J(\theta) = \mathbb{E}[(R(q, o) - b(q)) \nabla \log \pi_\theta(o \mid q)]. \tag{50}$$

Step D: Replace learned baseline by group baseline.

$$b(q_j) \approx \mu_j = \frac{1}{G} \sum_{i=1}^G R(q_j, o_{j,i}). \tag{51}$$

Step E: Normalize.

$$A_{j,i} = \frac{R(q_j, o_{j,i}) - \mu_j}{\sigma_j}. \quad (52)$$

Step F: Use PPO-style ratio and clipping.

$$\rho_{j,i,t}(\theta) = \frac{\pi_{\theta}(y_{j,i,t} \mid x_j, y_{j,i,<t})}{\pi_{\theta_{\text{old}}}(y_{j,i,t} \mid x_j, y_{j,i,<t})}. \quad (53)$$

$$\ell_{j,i,t}^{\text{clip}} = \min(\rho_{j,i,t} A_{j,i}, \text{clip}(\rho_{j,i,t}, 1 - \varepsilon, 1 + \varepsilon) A_{j,i}). \quad (54)$$

Step G: Add KL regularization.

$$\hat{J}_{\text{GRPO}} = \frac{1}{BG} \sum_{j=1}^B \sum_{i=1}^G \frac{1}{T_{j,i}} \sum_{t=1}^{T_{j,i}} (\ell_{j,i,t}^{\text{clip}} - \beta d_{j,i,t}^{\text{KL}}). \quad (55)$$

16 Suggested in-class questions

1. Suppose all $G = 16$ completions for a prompt receive reward 0. What are the advantages? What update signal remains?
2. Suppose one completion is correct and the other 15 are wrong. Which completion receives positive advantage? What happens to the incorrect completions?
3. Why is it useful to normalize rewards within each prompt instead of across the whole batch?
4. What is the difference between $\pi_{\theta_{\text{old}}}$ and π_{ref} ?
5. Why does removing the critic simplify training? What cost does GRPO add instead?
6. Why might R1-Zero produce strong reasoning but poor readability?

17 Takeaways

1. DeepSeek-R1-Zero is a clean demonstration of reasoning-oriented RL directly from a base model, without a cold-start SFT stage.
2. GRPO samples multiple completions for the same prompt and converts their rewards into group-relative advantages.
3. The method removes the learned critic/value model used in PPO-style actor-critic RL.
4. The training signal is simple: increase token probabilities for completions that outperform sibling completions, decrease them for completions that underperform, and regularize the policy with KL.
5. Rule-based rewards make the pipeline simpler, especially for math and code, but limit the method to tasks where correctness is checkable.
6. DeepSeek-R1 extends R1-Zero with cold-start SFT, rejection sampling, further SFT, and final RL for a more readable and general assistant.

References

- [1] Daya Guo et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948, 2025. <https://arxiv.org/abs/2501.12948>
- [2] Daya Guo et al. *DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning*. Nature, 2025. <https://www.nature.com/articles/s41586-025-09422-z>
- [3] Zhihong Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024. <https://arxiv.org/abs/2402.03300>
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>